CS410 TEXT INFORMATION SYSTEMS · FALL 2026 · TECHNOLOGY REVIEW

Can a Decision Model Rerank?

Decision models versus cross-encoder rerankers on ranking quality, cost, and latency

---

Dongkyu Kim · NetID: dongkyu3

Code, raw per-query results and analysis scripts are in the project repository (`results_final/`).

Question, approach, and thesis

Research question

A *decision model* returns a typed answer with probabilities instead of generated text. Can such a model serve as the second-stage reranker of a retrieval pipeline — and at what quality, cost, and latency compared with dedicated cross-encoder rerankers?

What I did

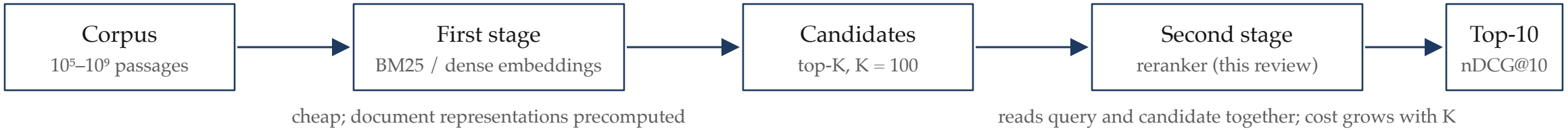
- Compared **7 systems** under identical conditions: 100 candidates per query, one seeded shuffle, one prompt per mode, no tuning.
- Two benchmarks with different label density: MIRACLReranking (2.2 positives/query) and JQaRAReranking (9.7 positives/query).
- Measured nDCG@10 (primary), billed cost per 1,000 queries, sequential and 64-way latency; paired bootstrap (10,000 resamples) for every comparison.
- Opened the black box: how Jev-style models are trained and called, and what their score distributions look like when used as rankers.

> **Thesis.** "Does the decision model replace the reranker?" is the wrong question. The same Jev 1.14 loses to Cohere Rerank 4-pro on MIRACL (0.750 vs 0.793) and wins on JQaRA (0.885 vs 0.819). The ranking depends on the data and on how the model is invoked, so the choice has to be measured on one's own data, with cost and latency in the loop.

Related review in the class library: "Tech Review: LLM Rerankers as a Second Stage over BM25" surveys LLM reranking methods; this review adds measurements and the mechanics of decision models.

BACKGROUND

Two-stage retrieval and the role of the reranker



Why a second stage

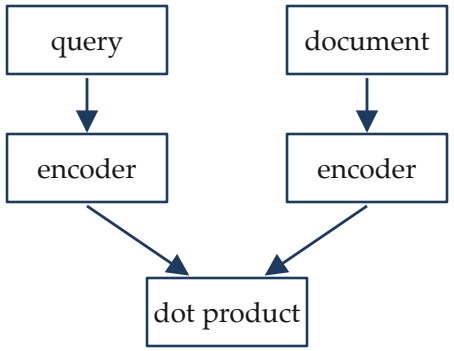
- The first stage scores query and document independently, so the index can be built offline and searched quickly — but it compares them only coarsely.
- The reranker reads the query together with each of the K candidates. Per-query cost is therefore proportional to K, and final quality is bounded by first-stage recall.
- BEIR reports that re-ranking and late-interaction models give the best zero-shot average, but at high computational cost; ColBERT attributes that cost to pushing every query–document pair through a large network.

What this implies for the comparison

- The unit of cost is the **candidate list** (100 passages per query), not a single document.
- Providers price it differently: per input token (decision models) or per search unit of up to 100 documents (Cohere) — so the cheaper option depends on passage length.
- Reference points in our samples: BM25 order gives nDCG@10 0.443 on MIRACL; a random order gives 0.114 on JQaRA.

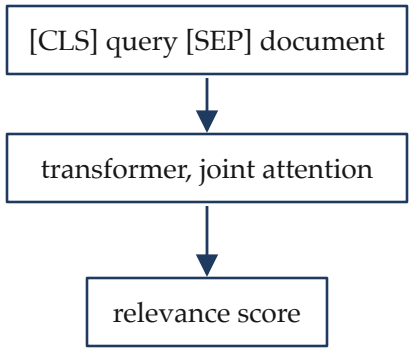
Cross-encoder rerankers

Bi-encoder (first stage)



document vectors precomputable

Cross-encoder (reranker)



one pass per pair; not precomputable

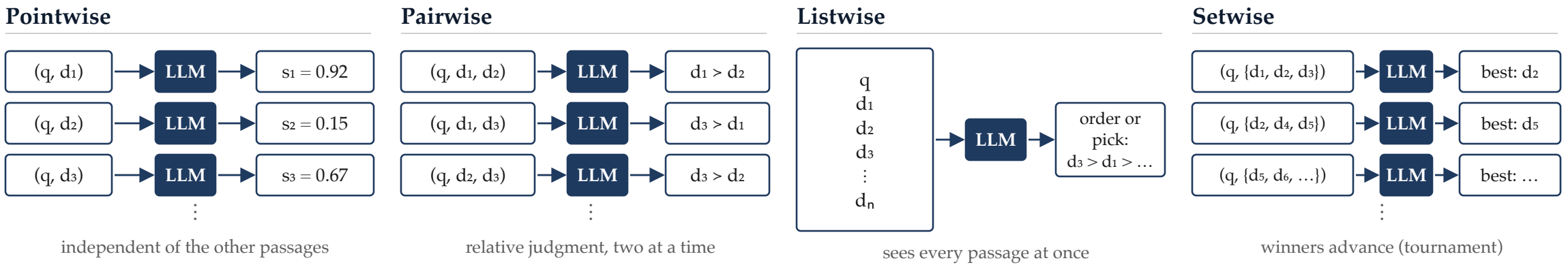
- Every query token can attend to every passage token, which is why cross-encoders are more accurate than independent encoders — and why they only run on the top-K.
- Scores are **pointwise**: each candidate is scored on its own, so the result does not depend on candidate order.

Model family	How the relevance score is produced
BERT reranker *Nogueira & Cho, 2019*	BERT fine-tuned on (query, passage) pairs; top of the MS MARCO leaderboard and state of the art on TREC-CAR at the time.
monoT5 *Nogueira et al., 2020*	T5 is trained to generate the target words "true" / "false"; the logits of those words are read as a relevance probability.
Qwen3-Reranker-8B *open weights, 32K context*	Decoder LLM with a chat prompt (`<Instruct> <Query> <Document>`) and a system rule "answer only yes or no". Score = softmax over the `no`/`yes` logits at the last position, i.e. P(yes) (model-card code).
Cohere Rerank *API, 100+ languages*	Returns `relevance_score` per document; billed per search unit. The documentation I read gives no architecture or training details — treated as a black-box pointwise scorer.

> **Bridge to decision models.** Qwen3-Reranker already turns an LLM into a ranker by reading a yes/no probability instead of generating text. Decision models expose that read-out as a typed API.

Nogueira & Cho (arXiv 1901.04085), Nogueira, Jiang & Lin (arXiv 2003.06713) — abstracts; Qwen3-Reranker-8B model card (code read directly); Cohere Rerank overview. Joint-input diagram follows the standard BERT setup.

Using LLMs as rerankers: four ways to prompt



Paradigm	LLM calls per query	Strength	Weakness	Reference
Pointwise	N, independent (parallel)	Independent of other candidates and of input order	No cross-passage comparison; scores need calibration	monoT5; Qwen3-Reranker
Pairwise	up to $O(N^2)$; linear variants exist	Relative judgments are easier than absolute scores	High call count and token overhead	PRP
Listwise	1 (long lists need chunking)	One pass; global comparison of all candidates	Context limit; order sensitivity; output must be parsed	RankGPT; RankZephyr
Setwise	fewer inferences than pairwise	Between pairwise accuracy and pointwise cost	Needs a sorting procedure over sets	Setwise paper

What the literature reports

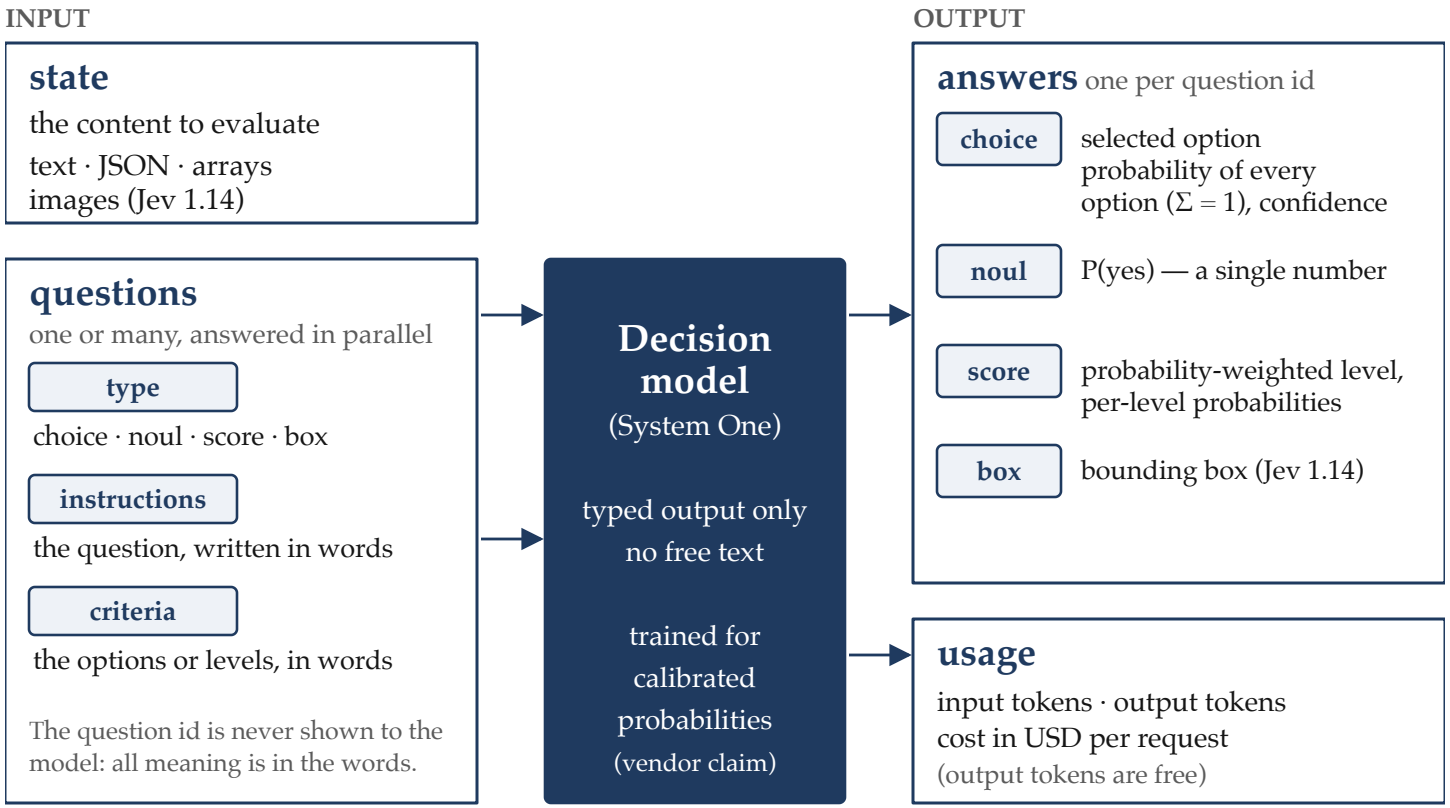
- Instructed LLMs can match supervised rankers (RankGPT); pairwise prompting (PRP) makes moderate-sized open models competitive.
- Pointwise is efficient but less effective; pairwise is effective but costly (Setwise paper).

Where decision models fit

- **Choice** over N options = listwise / setwise selection with a probability for *every* option.
- **Noul** = pointwise P(yes); many Noul questions can share one request.

Sun et al., RankGPT (arXiv 2304.09542); Qin et al., PRP (2306.17563); Zhuang et al., Setwise (2310.09497); Pradeep et al., RankZephyr (2312.02724) — abstracts only; diagrams and call-count column are my summary.

Decision models (System One): what goes in, what comes out



Not an LLM that writes answers *(vendor documentation; not independently verified)*

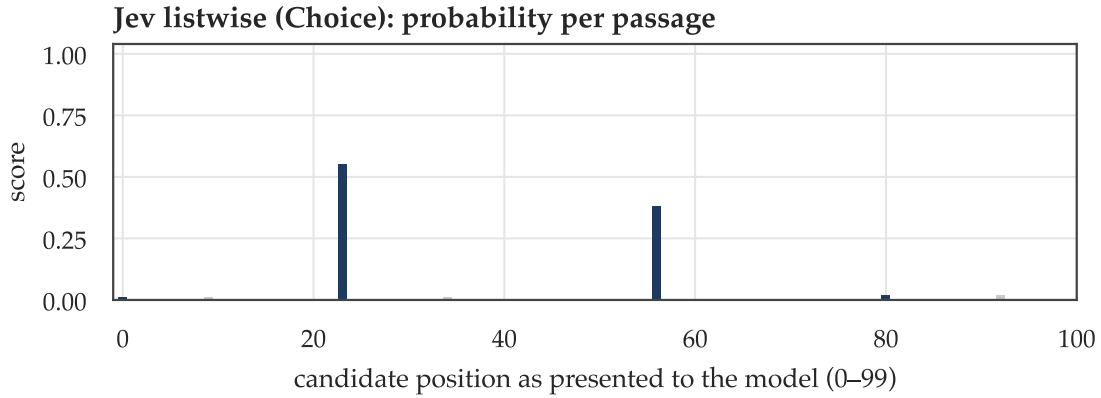
- Returns a **typed answer with probabilities**; the caller fixes the answer space. No generated text.
- Trained with **RLCD** — reinforcement learning for *calibrated decisions* — rather than RLHF or RLVR. Calibration holds across many predictions, not for a single answer.
- Questions in one request share a state that is read once and are evaluated in parallel.
- Jev 1.14: **$0.042 / M input tokens**, output free; 262K context on OpenRouter.
- Architecture, size and training data are **not disclosed** in the documents I read.
- One API, 22 models from 13 makers — I switched Jev → `openai/gpt-6-luna-decisions` ($0.10 / M) by changing the model name.

> **My observation.** gpt-6-luna reports `output_tokens = 0`: consistent with reading probabilities out of one forward pass, like Qwen3-Reranker's yes/no logit. An inference, not documented.

Turning a decision model into a reranker: two invocation modes

Listwise: one Choice question over all 100 passages

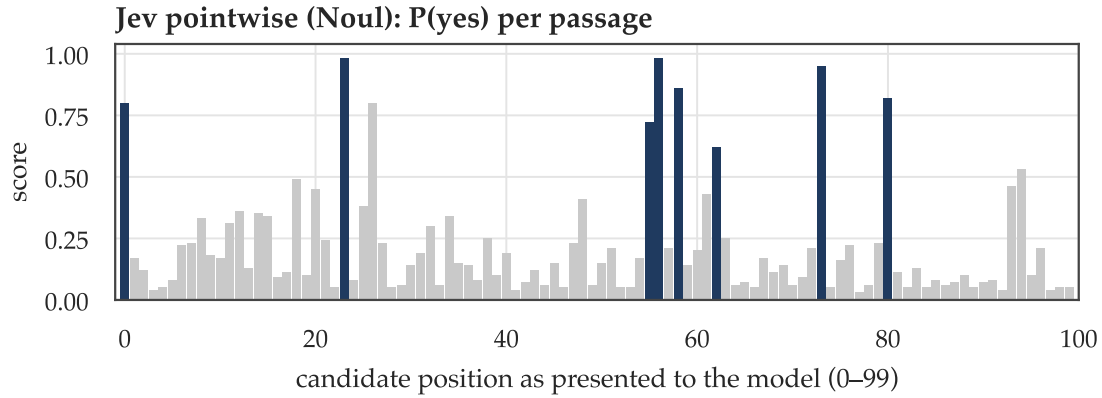
query in the state; 100 passages = the options of ONE Choice question → **Decision model** → probability over the 100 options ($\Sigma = 1$) $score(d_i) = P(option\ c_i)$



- Only **~6 candidates** (Jev) or **~20** (luna) get non-zero mass; the rest tie at 0 and keep the presented order.

Batched pointwise: 100 Noul questions in one request

query + 100 passages in the state; 100 Noul questions: "Is passage c_i relevant?" → **Decision model** → P(yes) for every passage, asked independently $score(d_i) = P(yes \mid c_i)$



- Every passage is scored, but at 0.01 resolution **~63% tie** (Jev); input tokens +18% (Jev) / +99% (luna) vs. listwise.

Bars: one real JQaRA query (`test_query831`), 100 candidates in the order the model received them; dark = labelled positive. All systems: same seeded shuffle, sort by score, ties broken by presented position, one fixed prompt per mode.

> **Not strictly pointwise.** TypeSafe's own re-ranking cookbook sends one request per (query, candidate) pair; mine puts all 100 passages in one state, so every question can see the whole list. I did not compare the two forms.

Experimental setup

	MIRACLReranking	JQaRAReranking
Queries	252 = 14 × 18 languages (seed 42); only queries with ≥1 positive in the pool	1,667 (full test split), Japanese
Candidates	BM25 top-100	top-100 from five embedding models fused by RRF
Positives / query	2.2	9.7
Labels	Relevance judgments by native speakers	Positive if the passage or title contains the answer string; part of the labels filtered by LLM answerability and human review
Passage text	Truncated per language to 218–533 characters (to fit Jev's context); identical text for all systems	Untruncated (about 400 characters or fewer)

Systems (7)

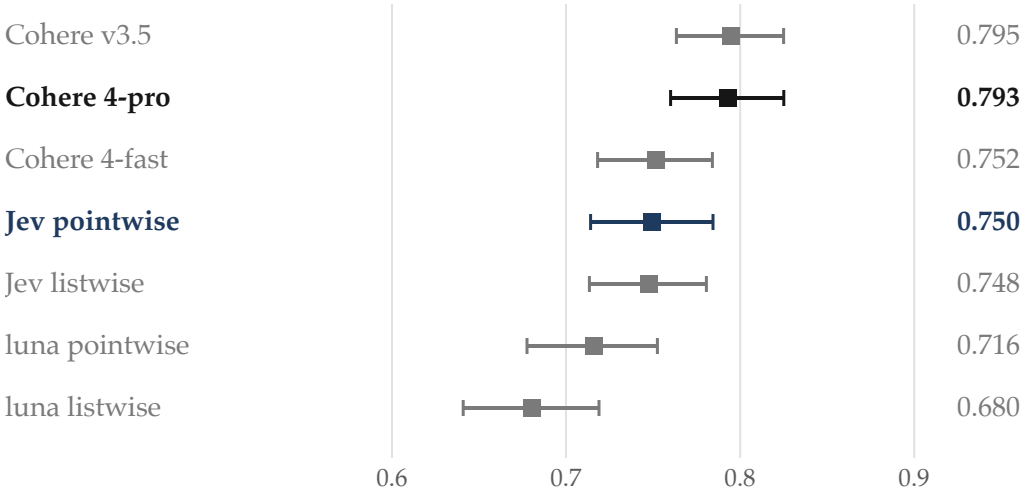
- Jev 1.14 `typesafe/jev-1.14`: listwise (Choice), pointwise (Noul)
- gpt-6-luna `openai/gpt-6-luna-decisions`: listwise, pointwise
- Cohere Rerank 4-pro, 4-fast, v3.5 via OpenRouter `/api/v1/rerank`
- **Excluded** after a 600-query probe: Voyage 2.5/3/lite (429 rate limits), hosted Qwen3-Reranker (503/429, slow), NVIDIA free tier (throttled)

Measurement protocol

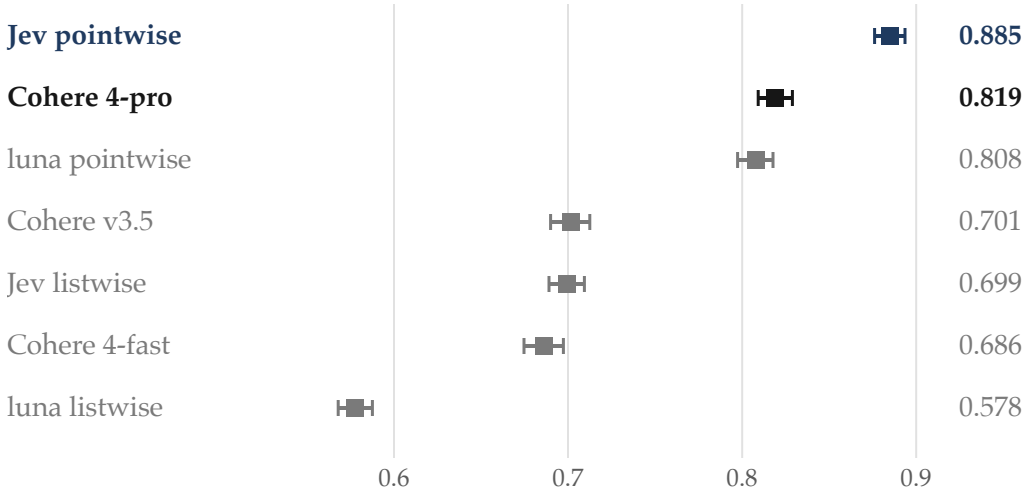
- **Quality:** nDCG@10 (primary), hit@1, MRR@10, recall@10; paired bootstrap over queries, 10,000 resamples, 95% CI.
- **Cost:** billed `usage.cost` from the API response; Cohere is per search unit, decision models per input token.
- **Latency:** service time of the successful HTTP call (harness queueing excluded), measured one request at a time (50 queries) and with 64 requests in flight (all queries).
- Failed queries (5 luna-listwise refusals) are scored on the presented order.

Ranking quality: the order of systems flips between datasets

MIRACLReranking · 252 queries



JQaRAReranking · 1,667 queries



Paired bootstrap on nDCG@10 (Δ mean [95% CI])	MIRACL	JQaRA
Jev pointwise − Cohere 4-pro	−0.044 [−0.068, −0.020]	+0.066 [+0.059, +0.073]
Jev pointwise − Jev listwise	+0.002 [−0.019, +0.022]	+0.186 [+0.177, +0.195]
Jev pointwise − luna pointwise	+0.034 [+0.007, +0.061]	+0.077 [+0.070, +0.084]
Jev listwise − luna listwise	+0.067 [+0.042, +0.092]	+0.121 [+0.113, +0.130]

Jev pointwise is 4th of 7 on MIRACL and 1st on JQaRA; Cohere 4-pro is the only system in the top two on both. Cohere v3.5 and 4-fast fall from 1st/3rd to 4th/6th. gpt-6-luna is below Jev in the same mode on both datasets. Reference: BM25 order 0.443 (MIRACL), random order 0.114 (JQaRA).

Price–performance and latency–performance: decision models vs. rerankers



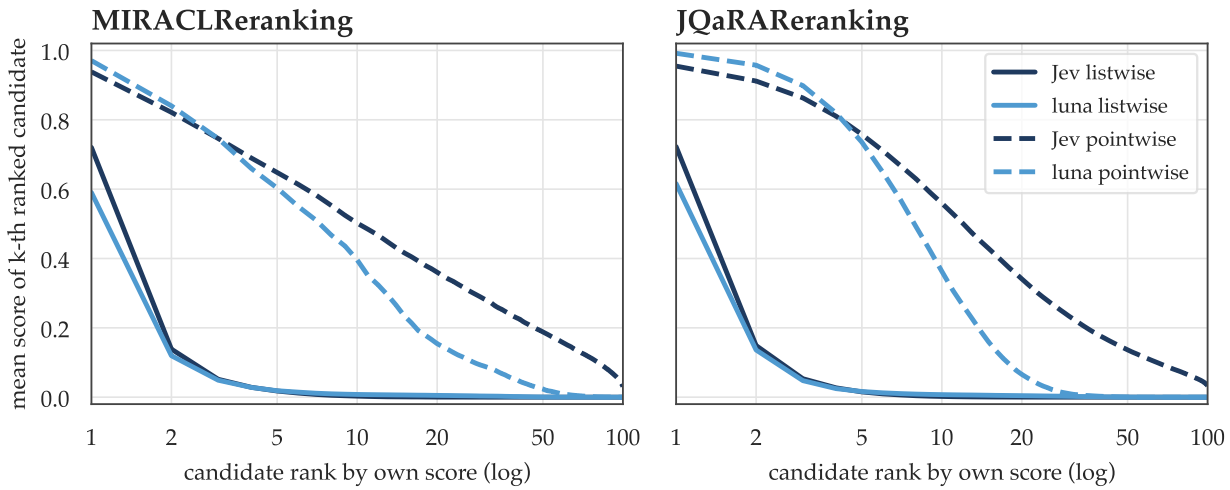
What the frontiers show

- **Frontier** (circled): Jev pointwise, Jev listwise and Cohere v3.5 in three of the four panels; v3.5 drops out of the JQaRA latency panel. Cohere 4-pro, 4-fast and gpt-6-luna are on no frontier.
- **JQaRA:** Jev pointwise beats 4-pro by 0.066 nDCG@10 and is $1.36 / 1K queries cheaper and 0.42 s faster — 4-pro is significantly dominated.
- **MIRACL:** 4-pro is tied with v3.5 at 2.5× the price. Against Jev pointwise: **$0.37 saved per 0.01 nDCG@10 given up** ($1.61 and 0.41 s saved for −0.044).
- **gpt-6-luna** is significantly dominated in all four panels, in both modes.
- **Under load** (64 in flight, JQaRA): Jev listwise slows to 2.78 s mean / 6.84 s p95; Cohere 4-pro stays at 1.05 s.

> **Cost model.** Decision models: input tokens × price (Jev $0.042/M, luna $0.10/M). Cohere: flat $0.0025 per query (4-pro). Break-even ≈ 60K (Jev) / 25K (luna) tokens per query; our 100-passage queries use 11–36K.

`results_final/FINAL.md`, `PARETO.md` (dominance uses paired bootstrap CIs); latency = mean service time, one request at a time; bars = 95% bootstrap CI of nDCG@10.

What the scores look like when a decision model is used as a ranker



System (mode)	Non-zero / 100 M / J	Ties M / J	Σ score M / J	Positives > 0 M / J
Jev · listwise	6.5 / 6.0	0.95 / 0.95	1.0 / 1.0	83% / 53%
luna · listwise	22.8 / 19.7	0.94 / 0.95	1.0 / 1.0	77% / 55%
Jev · pointwise	100 / 100	0.65 / 0.63	24.1 / 22.0	100% / 100%
luna · pointwise	22.3 / 15.4	0.85 / 0.89	10.9 / 8.9	92% / 82%
Positives per query	2.2 / 9.7			

M = MIRACL, J = JQaRA. Ties = 1 − (distinct scores / 100).

Listwise (Choice)

- Probability mass 1 sits on **about six** of 100 candidates (Jev); the other ~94 tie at zero and stay in the shuffled order.
- An excellent top-1 selector — JQaRA hit@1: luna 0.939, Jev 0.935, first and second of all systems — with no ordering information below it.
- With 9.7 positives per query, Jev gives non-zero mass to only 53% of them.

Pointwise (Noul)

- Jev scores all 100 candidates, but at 0.01 resolution about 63% tie; ties are broken by presented position.
- Σ P(yes) is 22–24 per query while qrels list 2.2 / 9.7 positives. The prompt's notion of relevance ("helps answer the question") is broader than the labels, so this is no proof of mis-calibration — but P(yes) should be used as a ranking score only.

Own analysis: `results_final/ANALYSIS.md` (section A) and per-query scores of the final runs (252 and 1,667 queries); chart: `make_figures.py`.

Why does the ranking flip? What is measured and what is not

Measured

Effect of invocation mode (nDCG@10)	MIRACL	JQaRA
Jev: pointwise − listwise	+0.002 [−0.019, +0.022]	+0.186 [+0.177, +0.195]
luna: pointwise − listwise	+0.035 [+0.005, +0.066]	+0.230 [+0.220, +0.240]
Positives per query	2.2	9.7
Jev listwise: positives with score > 0	83%	53%
Jev listwise hit@1	0.675	0.935
Jev pointwise hit@1	0.671	0.930

- The gap between modes is negligible on MIRACL and large on JQaRA; at rank 1 the two modes are about equal.
- Within one vendor, tiers differ by dataset: Cohere 4-pro 0.819 vs. 4-fast 0.686 and v3.5 0.701 on JQaRA, while on MIRACL 4-pro and v3.5 are tied (0.793, 0.795) and 4-fast is 0.752.

Confounded — not separated by this design

- **Label density:** 2.2 vs. 9.7 positives per query. Shallow listwise support costs more when there are more positives to rank.
- **Input truncation:** MIRACL passages were cut to 218–533 characters, JQaRA passages were not.
- **Label semantics:** JQaRA marks a passage positive when it contains the answer string (plus filtering); MIRACL uses graded human relevance judgments.
- **Language and domain:** 18 languages of Wikipedia queries vs. Japanese quiz questions.

Hypotheses (untested)

- Quiz questions are answerable from world knowledge, which may help a model judge whether a passage contains the answer.
- Vendor documentation says Jev is weaker outside English, yet it is first on Japanese JQaRA — unexplained.
- Possible contamination of any model with JAQKET/Wikipedia material cannot be excluded.

`results_final/FINAL.md`, `ANALYSIS.md`; JQaRA label construction from the dataset card; vendor language statement from TypeSafe "Models" documentation.

Limitations and negative results

Threats to validity

- **One run per system and dataset.** Re-running MIRACL changed scores by −0.006 to +0.002 (e.g. Jev pointwise 0.7552 → 0.7496). Latency variance across runs was not measured.
- **Long-context mode.** Passage text exceeded 32,000 bytes for 92% (MIRACL) and 97% (JQaRA) of queries. OpenRouter's Jev guide says such requests run in a slower, less accurate long-context mode (vendor claim, unverified); if true, Jev's scores are conservative.
- **One prompt per mode, no tuning;** no threshold or instruction search for any system.
- **"Pointwise" is shared-context** (all passages in one state); per-pair requests were not compared. The ordinal **Score** primitive was not tried.
- Cohere Rerank 5, named in Cohere's documentation, was not in the OpenRouter catalog I used.
- Costs and latencies are via OpenRouter, not vendor endpoints; results come from a single day.

Negative and unexpected results

- **Rank fusion does not help.** Reciprocal rank fusion of two systems was significantly worse than the better member on JQaRA in all six pairs tried (e.g. Jev pointwise + Cohere 4-pro: −0.015 [−0.019, −0.010]) and indistinguishable from it on MIRACL. A hybrid pipeline is not supported by this data.
- **Refusals.** For 5 of 1,667 JQaRA queries, gpt-6-luna in listwise mode returned "OpenAI refused to answer" on every retry; those queries were scored on the presented order. Cohere returned a score for every query.
- **Exclusions.** Voyage rerank models returned HTTP 429 at sustained volume (2,237 times for the lite model over 1,667 queries) and hosted Qwen3-Reranker was slow or unavailable; their quality is not in the final table.
- **Qrels are incomplete.** On MIRACL, 47 of 53 candidates that Jev scored ≥0.5 but the qrels called non-relevant were judged relevant by an independent LLM judge (phase-1 data, Jev-only) — absolute scores understate recall of true relevance.

`results_final/ANALYSIS.md` (B, C); `results_jqara/SUMMARY.md` (probe, refusals); OpenRouter Jev guide (long-context note); phase-1 judge run: `results/judge_relevance_full.jsonl`.

Conclusion: what I learned

- **Rankings do not transfer between datasets.** The same Jev 1.14 loses to Cohere 4-pro on MIRACL (0.750 vs 0.793) and wins on JQaRA (0.885 vs 0.819).
- **A decision model is a score-and-select API, not a drop-in cross-encoder.** Listwise piles probability on ~6 of 100 passages (great top-1, shallow ranking); pointwise scores everything but ties a lot; P(yes) is a ranking score, not the probability of relevance.
- **Price structure matters more than list price.** Tokens × price versus a flat per-query fee: Jev sits on the cost frontier on both datasets, Cohere 4-pro and gpt-6-luna never do.
- **Measure it yourself, and measure carefully.** Client-side queueing had inflated my first latency numbers, rate limits removed Voyage, and one run per system is a real limit. Rank fusion of two systems did not help.
- **Practical rule:** A/B on 50–200 of your own queries, with cost and latency in the table.

Resources I found most useful

Resource	Source	Why it is useful
Passage Re-ranking with BERT	Nogueira & Cho, arXiv 1901.04085	Origin of cross-encoder reranking; MS MARCO state of the art.
Pretrained Transformers for Text Ranking: BERT and Beyond	Lin, Nogueira & Yates, arXiv 2010.06467	Survey of transformer rerankers and the effectiveness–efficiency trade-off.
Document Ranking with a Pretrained Sequence-to-Sequence Model	monoT5, arXiv 2003.06713	Reads "true/false" logits as relevance — ancestor of yes/no scoring.
Is ChatGPT Good at Search? (RankGPT)	Sun et al., arXiv 2304.09542	Listwise LLM reranking and permutation distillation.
LLMs are Effective Text Rankers with Pairwise Ranking Prompting	Qin et al., arXiv 2306.17563	Why pointwise and listwise prompts are hard; pairwise remedy.
A Setwise Approach for Zero-shot Ranking with LLMs	Zhuang et al., arXiv 2310.09497	Compares the three paradigms on effectiveness and cost.
RankZephyr	Pradeep et al., arXiv 2312.02724	Open listwise reranker; robustness to input order.
ColBERT; BEIR	arXiv 2004.12832; 2104.08663	Cost of cross-encoding; zero-shot evaluation of re-ranking.
Making a MIRACL; JQaRA dataset card	arXiv 2210.09984; hotchpotch/JQaRA (Hugging Face)	The two benchmarks, including how labels were built.
Qwen3-Reranker-8B model card	Hugging Face; arXiv 2506.05176	Open yes/no-logit reranker with runnable code.
Cohere Rerank overview	docs.cohere.com	API behaviour and search-unit billing.
TypeSafe docs (System One, AI primer, Noul, Choice, Re-ranking cookbook)	docs.typesafe.ai	Primary source for decision models; vendor claims, read critically.
OpenRouter: Jev guide and "What is Jev?"	openrouter.ai/docs, openrouter.ai/blog	Decisions API, other decision models, pricing, context notes.
This project's repository	code, raw per-query results, analysis scripts	Reproduces every number in this deck (`results_final/`).

Papers checked via arXiv abstracts; vendor documentation (TypeSafe, OpenRouter, Cohere, Qwen) read for the cited pages and treated as claims, not evidence. Links opened 2026-11.